

TP5 — Deep Learning

Autoencoders, Denoising Autoencoders y Variational Autoencoders (numpy puro)

Sistemas de Inteligencia Artificial — ITBA

Problema y dataset

EJ1.a — Autoencoder basico

EJ1.b — Denoising Autoencoder

EJ2 — Variational Autoencoder

Conclusiones

Problema y dataset

El problema

- Implementar **Autoencoders** en **numpy puro**, sin frameworks de DL.
- Reusamos la maquinaria de MLP con backpropagation de TP3 (forward/backward, optimizadores, activaciones, inicializadores).
- Lo nuevo del Deep Learning: BCE numericamente estable, inicializacion segun la activacion (Glorot/He) y la derivacion completa del VAE.

Tres bloques

1. **EJ1.a** — Autoencoder basico (cuello 2D, error ≤ 1 pixel).
2. **EJ1.b** — Denoising Autoencoder.
3. **EJ2** — Variational Autoencoder sobre emojis.

- 32 caracteres de una fuente de 7 columnas (“ ‘ ” a DEL).
- Cada caracter es una grilla $7 \times 5 = 35$ **pixeles** binarios $\{0, 1\}$.
- Se decodifica de hexadecimal a una matriz binaria ($X \in \{0, 1\}^{32 \times 35}$).
- Es un dataset chico y discreto: ideal para forzar al autoencoder a *memorizar* a traves de un cuello de botella.

Letra 'a':

```
.....  
....#  
.####  
#...#  
#..##  
.##.#  
.....
```

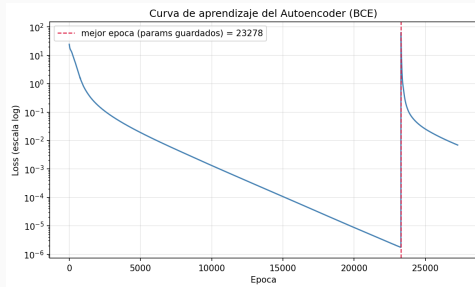
EJ1.a — Autoencoder basico

Arquitectura del AE basico

- Arquitectura simetrica con **cuello de botella 2D**:

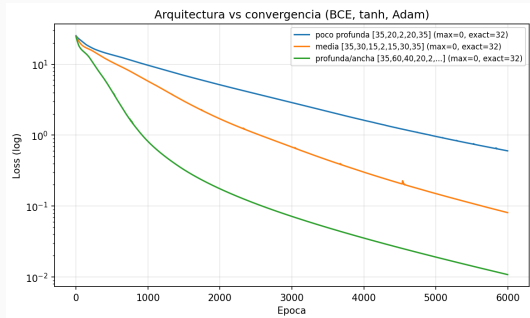
$35 \rightarrow 60 \rightarrow 40 \rightarrow 20 \rightarrow 2 \rightarrow 20 \rightarrow 40 \rightarrow 60 \rightarrow 35$

- Activaciones: **tanh** en las ocultas, latente *lineal*, salida **logistica** (píxeles en $(0, 1)$).
- Perdida **BCE** (binaria), **Adam** ($lr = 10^{-3}$), full-batch, EarlyStopping.
- **Sin regularizacion**: queremos sobreajustar (memorizar) las 32 letras.

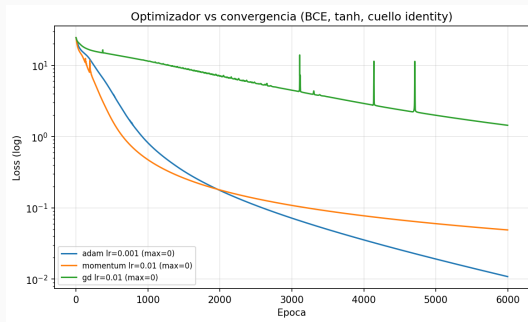


Curva de loss (BCE) durante el entrenamiento.

Busqueda de arquitecturas y optimizadores



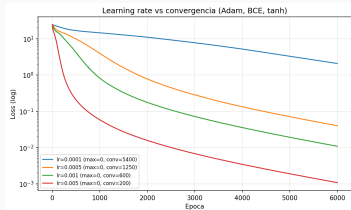
Profundidad/anchura vs convergencia.



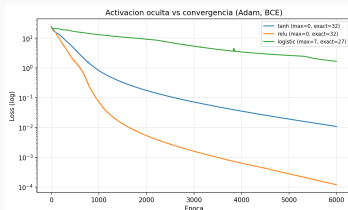
GD vs Momentum vs Adam.

Epoca de convergencia (primera con $\max \leq 1$): Adam @600 $\ll$ Momentum @500 $\ll$ GD @4300. Las tres arquitecturas llegan a $\max=0$; la profunda-ancha converge mas rapido (@600).

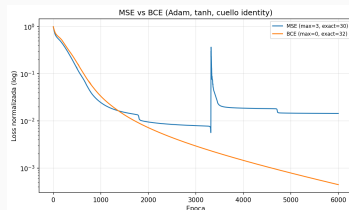
Learning rate, activacion y loss



Learning rate (Adam).



Activacion oculta.

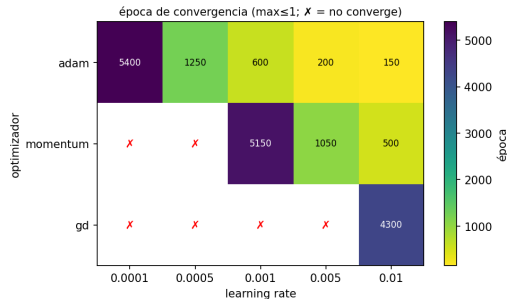
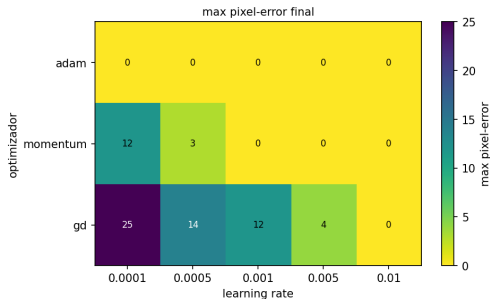


BCE vs MSE.

tanh y **relu** convergen a max= 0; la **logistica** oculta *no* converge (max= 7, 27/32). **BCE** llega a max= 0 @600; **MSE** se queda en max= 3 (30/32) en 6000 ep.

Interaccion optimizador × learning rate

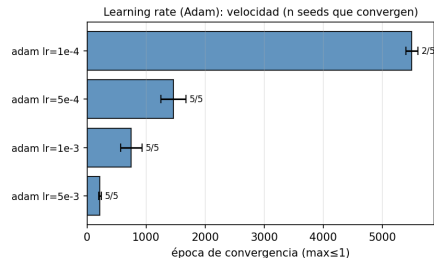
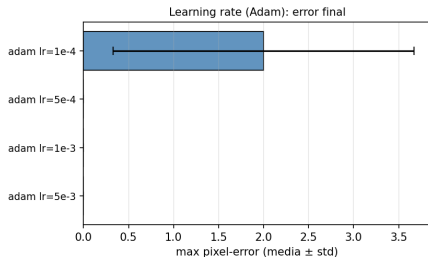
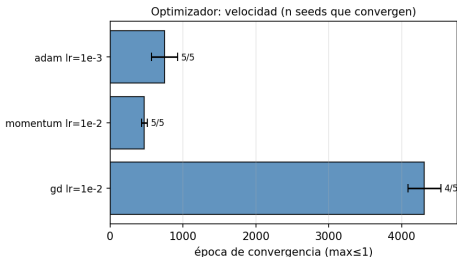
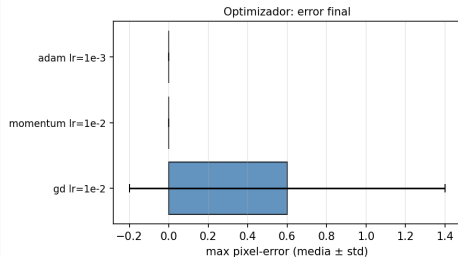
Interacción optimizador × learning rate (6000 épocas, seed=0, BCE)



El lr optimo *depende* del optimizador: **Adam** (paso adaptativo por parametro) converge a max= 0 en *los cinco* lr probados (de @5400 con 10^{-4} a @150 con 10^{-2}); **Momentum** solo con $lr \geq 10^{-3}$; **GD** solo con $lr = 10^{-2}$ (@4300). Barrer lr y optimizador por separado puede enganar: la grilla completa muestra que la robustez al lr es la ventaja real de Adam.

Variabilidad entre seeds (las conclusiones no son suerte)

Variabilidad entre 5 seeds (arquitectura ganadora, BCE, 6000 épocas)



Resultado: criterio ≤ 1 pixel CUMPLIDO

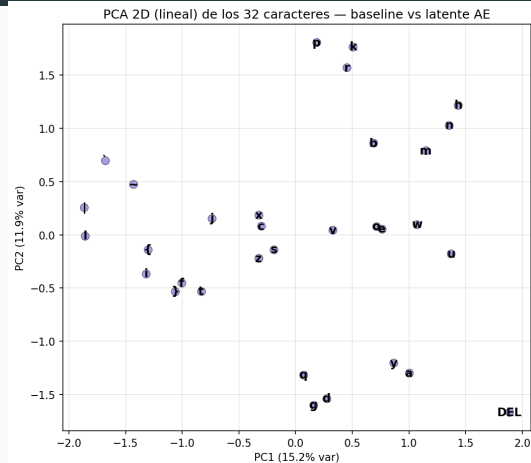
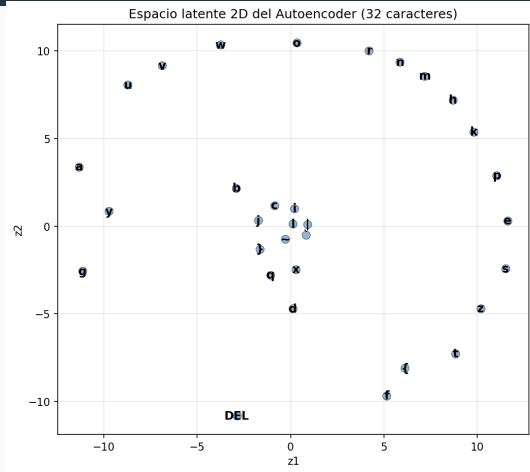
Metrica	Valor
max pixel-error	0
mean pixel-error	0.0000
letras exactas	32/32
letras con ≤ 1	32/32
BCE final (mejor ep)	$1,74 \times 10^{-6}$

Las 5 seeds [0, 1, 2, 3, 42] llegan a max= 0 y 32/32 exactas. El modelo recargado desde model.npz reproduce el resultado.



Original vs reconstruccion (32 letras).

Espacio latente 2D vs PCA lineal

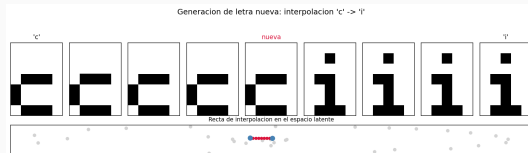


PCA 2D captura solo 27,1 % de la varianza (PC1 15,2 % + PC2 11,9 %), mientras que el AE *no lineal* con el mismo cuello 2D reconstruye perfecto. El histograma de pixel-error confirma 32/32 exactas.

Generacion de una letra nueva

- Tomamos un punto *nuevo* del espacio latente 2D y lo pasamos por el decoder.
- El AE genera una "letra" que mezcla rasgos de los vecinos latentes.
- Limitacion: con un cuello tan chico (2D) y sin un prior, gran parte del latente son *agujeros* que decodifican a basura.

Esta limitacion motiva el VAE (EJ2): un latente *continuo y muestreable* con un prior gaussiano.

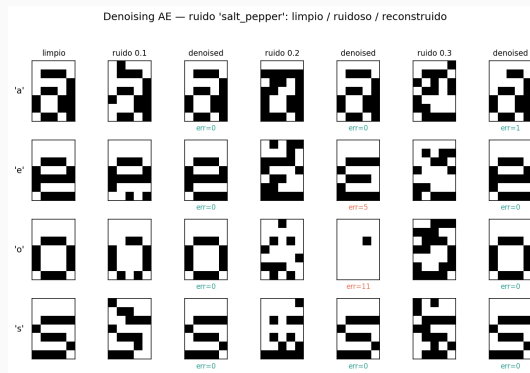


Decodificacion de puntos latentes nuevos.

EJ1.b — Denoising Autoencoder

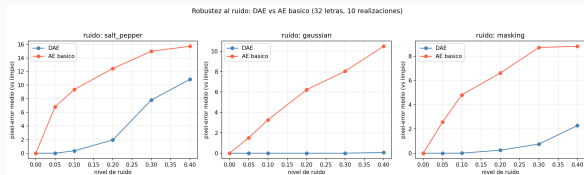
Denoising: idea y ruido

- Entrenamos un DAE con **augmentation online**: cada epoca se regenera el ruido y el target es la letra *limpia*.
- Ruido de entrenamiento: **sal-y-pimienta** $p = 0,10$.
- Misma arquitectura simetrica; cuello un poco mas ancho (8D, el denoising no exige 2D).
- Se evalua frente a sal-y-pimienta, gaussiano y masking.



Ejemplos: ruidoso → limpio (DAE).

Barrido de niveles de ruido

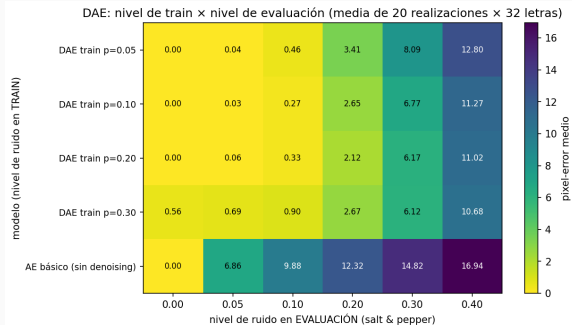


p (s&p)	DAE	AE basico
0.05	0.027	6.86
0.10	0.267	9.88
0.20	2.65	12.32
0.30	6.77	14.82
0.40	11.27	16.94

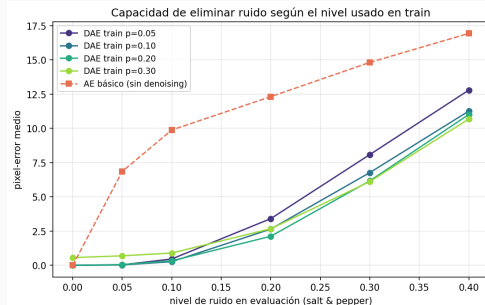
pixel-error *medio* (32 letras). El DAE reduce el error
 $\sim 25\text{--}250\times$ en niveles bajos/medios.

Generaliza a ruido no visto: gaussiano ($\sigma=0,4$: 0.15) y
masking (frac= 0,4: 2.12).

Nivel de ruido de *train*: trade-off



Un DAE entrenado por nivel p (12000 ep c/u) $\times$ nivel de evaluación.



Curva por DAE; baseline AE basico punteado.

Mas ruido en train $\Rightarrow$ mas robustez en niveles altos (eval 0,30: 6,12 con train $p=0,30$ vs 8,09 con $p=0,05$) pero se degrada la entrada *limpia* (err 0,56 con $p=0,30$). Sweet spot: $p \in [0,10, 0,20]$ (limpio perfecto, robustez casi maxima). Cualquier DAE $\ll$ AE basico en todo nivel.

EJ2 — Variational Autoencoder

Motivacion: los "agujeros" del latente

- Un AE comun aprende un latente *arbitrario*: puntos aislados con grandes regiones vacias que decodifican a ruido.
- No hay forma principiada de **muestrear** letras/imagenes nuevas.

El VAE: un latente continuo y muestreable

- El encoder produce una **distribucion** $q(z|x) = \mathcal{N}(\mu, \sigma^2)$ en lugar de un punto.
- Un termino **KL** la empuja hacia el prior $\mathcal{N}(0, I)$, llenando los agujeros.
- Asi podemos generar muestreando $z \sim \mathcal{N}(0, I)$ y decodificando.

Teoría: ELBO, KL y reparametrización

Maximizamos el **ELBO** (equivalente a minimizar):

$$\mathcal{L} = \underbrace{\mathbb{E}_{q(z|x)}[-\log p(x|z)]}_{\text{reconstrucción (BCE)}} + \beta \cdot \underbrace{D_{\text{KL}}(q(z|x) \parallel p(z))}_{\text{regularización}}$$

- **KL cerrado** (prior gaussiano): $D_{\text{KL}} = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$.
- **Reparametrización** (gradiente a través del muestreo): $z = \mu + \sigma \odot \epsilon$, con $\epsilon \sim \mathcal{N}(0, I)$.
- β pondera el KL (β -VAE): controla el trade-off reconstrucción vs estructura del latente.

Gradient-check

Backprop analítico vs diferencias finitas: error relativo máximo $1,25 \times 10^{-10}$ (tol 10^{-4}) —

PASA.

Dataset de emojis

- 16 clases de emojis (caras, corazon, estrella, sol, etc.).
- Rasterizados desde **NotoColorEmoji** a **16×16** en escala de grises (input_dim = 256).
- Augmentation (escala, rotacion, traslacion) → **960 muestras** (60 por clase), determinista (seed=42).



Una muestra por clase.

VAE: entrenamiento y reconstrucción



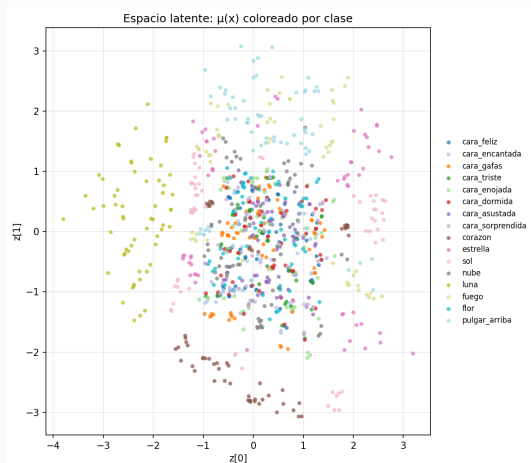
Loss total = reconstrucción + KL.



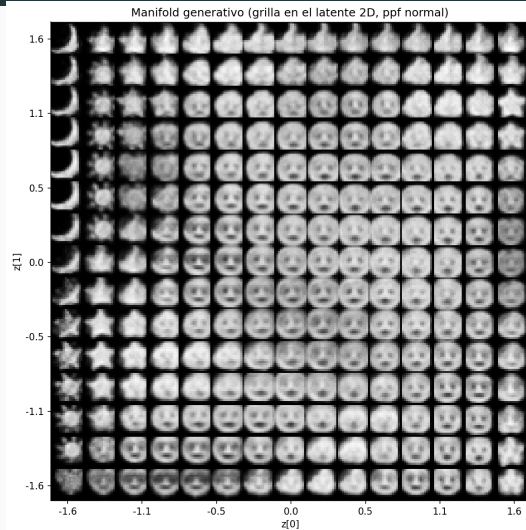
Original vs reconstrucción ($z = \mu$).

Principal (latent_dim=2, BCE, $\beta=1$, 600 ep): loss total = 117,99 (rec 112,22 + KL 5,78); MSE de reconstrucción = 4,58; salidas en $[0, 0,97]$, sin NaN.

VAE: espacio latente y grilla generativa



Latente 2D coloreado por clase.



Manifold: decodificación de una grilla de z .

El prior gaussiano *compacta* el latente: la grilla genera transiciones suaves entre emojis (sin agujeros como en el

VAE: muestreo e interpolacion

Muestras nuevas $z \sim \mathcal{N}(0, I)$ decodificadas



Muestras $z \sim \mathcal{N}(0, I)$.

Interpolación en el latente: corazon → pulgar_arriba

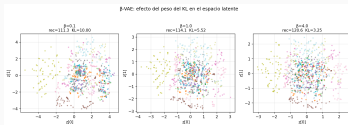


Interpolacion lineal en el latente.

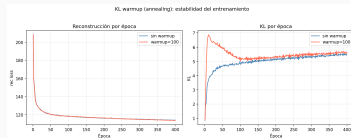
VAE: experimentos (latent_dim, β , KL warmup)



latent_dim.



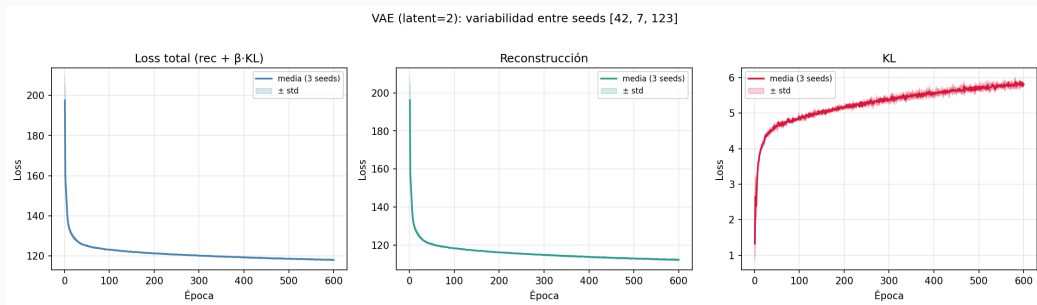
β -VAE.



KL warmup.

latent_dim: dim 2 (rec_MSE 5.27) $\rightarrow$ dim 8/16 (rec_MSE ≈ 1.6) reconstruyen mucho mejor. β : 0,1 (rec 111.3 / KL 10.0) vs 4,0 (rec 120.6 / KL 3.25): trade-off rec/KL clasico. **warmup** (100 ep): reconstruccion final algo mejor (113.6 vs 114.1).

VAE: variabilidad entre seeds



El entrenamiento es doblemente estocastico (init de pesos + sampleo de ϵ en la reparametrización); 3 seeds [42, 7, 123] $\times$ 600 épocas dan curvas casi indistinguibles: loss total final $118,02 \pm 0,14$ ($< 0,3\%$ de dispersión), rec $112,2 \pm 0,2$, KL $5,81 \pm 0,03$, rec_MSE 4,45–4,69. Además, seed= 42 reproduce *exactamente* la corrida principal (determinismo del pipeline).

Conclusiones

- **EJ1.a:** el AE no lineal con cuello **2D** memoriza las 32 letras con **0 píxeles de error** (32/32 exactas), muy por encima del baseline PCA 2D (27 % de varianza).
- **EJ1.b:** el DAE limpia ruido entre $\sim 25\text{--}250\times$ mejor que el AE básico y **generaliza a ruido no visto** (gaussiano, masking).
- **EJ2:** el VAE pasa el gradient-check ($1,25 \times 10^{-10}$), aprende un latente *continuo* y permite **generar** emojis nuevos por muestreo e interpolación.

- **Cuello 2D**: la reconstrucción perfecta del AE básico es *memorización* de 32 patrones; con datos más ricos un cuello 2D no alcanza (lo vemos en el VAE: dim 2 reconstruye peor que dim 8/16).
- El **AE común** no tiene un latente muestreable (agujeros); el VAE lo resuelve, pero a costa de algo de nitidez (trade-off β).
- Dataset chico y de baja resolución (font.h 7×5 , emojis 16×16): buen banco de pruebas pedagógico, no representativo de imágenes naturales.

Gracias.